os lab2

林家立 241220068

2961184218@qq.com

一.实验进程

已完成 lab2 要求的全部内容，包括系统调用、RTC 读时、时钟中断、用户态输出封装以及应用层异常测试。程序能够在用户态显示实时时钟，并能通过越界访问和特权指令测试验证内核捕获异常的能力。

二.实验结果

程序运行后，左上角能够显示两行内容：`<Real Time Clock>` 和 `Current Time: HH:mm:ss`，并且时间每秒动态刷新一次。

执行用户态越界访问后，内核能够捕获通用保护异常，并打印对应的异常信息，说明分段保护生效。

地址访问越界：

```
<bochs:1> c
Kernel initialized.
Loading application module...

--- General Protection Fault Caught by Kernel ---
Application tried to access restricted memory or performed invalid operation.
Trap Frame Info:
  IRQ: d, Error Code: 0
  EIP: 6e6, CS: 1b, CPL: 3
  DS: 10023, ES: 10023, SS: 23
Confirmed: Exception occurred in USER MODE (CPL 3).
Segmentation protection is ACTIVE.
Halting system.
```

越权:

```
gary@gary-VMware-Virtual-Platform:~/oslab/lab2-241220068/lab2$ make run
qemu-system-i386 -drive format=raw,file=os.img -rtc base=localtime -serial stdio
Kernel initialized.
Loading application module...

--- General Protection Fault Caught by Kernel ---
Application tried to access restricted memory or performed invalid operation.
Trap Frame Info:
  IRQ: d, Error Code: 0
  EIP: 6f0, CS: 1b, CPL: 3
  DS: 23, ES: 23, SS: 23
Confirmed: Exception occurred in USER MODE (CPL 3).
Segmentation protection is ACTIVE.
Halting system.
```

三.实验修改代码位置

1. /kernel/x86/idt.c

添加系统调用门和时钟中断门，使 `int 0x80` 与 `irqTimer` 能正确进入内核。

2. /kernel/device/timer.asm

配置 8253 定时器为 100Hz 周期中断，为 `sleep()` 和系统 tick 计数提供基础。

3. /kernel/x86/gdt.c

实现 `enterUserSpace`，设置用户段选择子并通过 `iret` 切换到 Ring 3；同时完成用户程序加载后的启动流程。

4. /kernel/x86/trap_entry.asm

实现系统调用入口和公共中断入口，统一保存/恢复现场并调用 `irqHandle`。

5. /kernel/core/irq_dispatch.c

实现系统调用分发、写屏、读取 RTC、时钟中断计数等核心逻辑。

6. /lib/doSyscall.asm

实现用户态系统调用封装，把参数装入寄存器并触发 `int 0x80`。

7. /lib/syscall.c

实现 `now()`、`sleep()` 和 `printf()`，完成用户态库封装。

8. /app/main.c

加入用户态越界访问和特权指令测试代码，用于验证异常捕获功能。

四.遇到的问题

1. syscallWrite 为什么要做段基址转换

一开始容易把用户传入的指针直接当作可访问地址，但实际上它只是用户段内的偏移。后来通过 `getSegBase(tf->ds)` 把段基址取出，再加上偏移，才能正确访问用户缓冲区。

五.思考题

1. 为什么用户程序不能直接调用内核函数，而要通过 `int 0x80` 这种特殊方式?

因为用户程序运行在 Ring 3，内核运行在 Ring 0。直接调用会绕过权限检查和现场保护，破坏隔离。`int 0x80` 由 CPU 受控陷入内核，能自动切换特权级并保存现场，安全且统一。

2. 当用户程序执行 `int 0x80` 触发系统调用时，CPU 自动将哪些内容压入栈中？返回时栈帧如何变化?

CPU 会把返回现场压栈，主要包括 EIP、CS、EFLAGS；若发生特权级切换，还会额外压入用户态的 SS 和 ESP。内核处理完后执行 `iret`，按相反顺序恢复这些寄存器并回到用户态。

3. 在 `syscallWrite` 函数中，为什么需要通过 `getSegBase(tf->ds)` 获取用户段基址?

因为用户态指针是段内偏移，不是内核直接可访问的物理地址。内核必须先取出用户段基址，再把基址和偏移相加，才能正确定位用户缓冲区并安全读取。

4. app 被 kernel 加载到内存后，为什么不能直接跳转执行？框架代码如何处理？

app 按用户段链接，直接跳转会缺少用户态段、用户栈和权限切换。框架先把 app 装入 APP_BASE，再由 `enterUserSpace` 构造 `iret` 现场切到 Ring 3，保证入口和栈都正确。